

Persona Onboarding: Technical Design

Technology choices, the tradeoffs behind them, and how the conversation engine works. Companion to the project plan. Prepared September 23, 2026.

1. Decisions at a glance

Layer	Choice	Status
Voice architecture	Cascaded: speech-to-text, then the LLM, then text-to-speech. Speech-to-speech is a stretch-goal toggle.	Decided
Turn-taking	Semantic turn detection plus backchannel filtering	Decided
The brain (LLM)	Claude Sonnet 5 at low effort, with prompt caching and streaming. Claude Haiku 4.5 as a latency fallback.	Decided
Dialogue management	Hybrid open slot filling plus an LLM (code owns the rules, the model owns the conversation)	Decided
Extracting information	Tool call inside the main response, validated by code	Decided
Frustration detection	The LLM's read of the words plus behavioral signals computed by code. Audio emotion analysis is a stretch goal.	Decided
Speech-to-text	Deepgram streaming with keyword boosting for names	Decided
Text-to-speech and voice	Decided by a structured voice bake-off: ElevenLabs vs. Cartesia, with OpenAI as a baseline	Pick by testing
Voice framework	Pipecat (Python) or LiveKit Agents (Python or Node.js)	Depends on your strongest language
Gmail	Real Google sign-in, read-only calendar and email access	Decided
Session storage	SQLite or Postgres, so sessions survive a server restart during the team's review	Decided

2. Why cascaded over hybrid

A **hybrid** setup would run the phone call on a speech-to-speech model (the model hears audio and speaks back directly) and run the text chat on a separate text model, with both saving to the same state. It's the flashier option. For this project, cascaded is the better choice because of four decisions already made:

Your requirement	Cascaded	Hybrid
Use Claude Sonnet 5	Sonnet 5 runs both the call and the chat	Not possible on the call. Claude has no speech-to-speech model, so the call would run on OpenAI or Google while the chat runs on Claude.
Test and choose between voices	Any voice from any provider (ElevenLabs, Cartesia, OpenAI, and others)	Only the speech-to-speech provider's small set of built-in voices

Your requirement	Cascaded	Hybrid
Semantic turn detection and backchannel filtering	You build and tune them, and can show them off in the README	Mostly handled inside the provider's API. Less to show, less control.
Survive hangups and channel switching	One brain, one conversation history. Switching from call to text is seamless.	Two brains that behave slightly differently. The handoff needs extra work to feel continuous.
Testing	Every turn is readable text, so automated evals are easy	Audio in, audio out. Voice turns are harder to test.
Latency	About 1 second with streaming at every stage	Best, often under 1 second
Hears tone of voice	Approximated with audio annotations and behavioral signals	Yes, natively

When hybrid would win: if latency and native emotional hearing mattered more than model choice, voice choice, and consistent behavior across channels. Here, they don't.

How to make cascaded feel cutting-edge

- Semantic turn detection and backchannel filtering (section 4). This is what reviewers will feel on the call.
- Streaming at every stage, so the agent starts speaking about a second after the user stops.
- Connecting Gmail on screen mid-call while the conversation keeps going (voice plus screen, not just voice).
- A documented voice bake-off with real data (section 8).
- Latency measured for every turn and reported in the README (section 10).
- Stretch goal: a speech-to-speech mode behind a toggle, with a short comparison in the README.

Decision: Cascaded. Speech-to-speech stays on the list as a stretch goal.

3. How the conversation engine works: hybrid open slot filling plus an LLM

3.1 The idea in plain terms

- **Slots** are the blanks you need filled: agent name, user name, Gmail, and what they need help with.
- **Open** means any blank can be filled at any moment, in any order. A form asks question 1, then 2, then 3. Open slot filling listens to everything and fills whatever it hears.
- **Hybrid** means two parts split the job. **Your code** holds the slot table and enforces the rules. **The LLM** understands what the user said, fills slots, and decides how to phrase things.

In research, this is called **mixed-initiative, frame-based dialogue management**. "Mixed-initiative" means either side can steer: the agent asks for what it needs, but the user can jump ahead, go off-topic, or bring things up on their own. It's a good term to use in the README.

3.2 Compared with the alternatives

Approach	How it works	What goes wrong
Rigid form (classic state machine)	A fixed script of steps: ask the name, wait, ask the next thing	Feels exactly like the form the spec tells you not to build. Breaks when users answer out of order.
Pure prompt (the LLM does everything)	One big prompt; the model decides everything	Drifts, forgets to ask for Gmail, offers graduation at random. You can't guarantee behavior under stress testing.
Hybrid open slot filling (chosen)	Code owns the slots and rules; the LLM owns understanding and wording	More moving parts, but reliable and natural at the same time

3.3 Who owns what

Your code owns	The LLM owns
The slot table: what's filled, what's missing, and what was corrected	Understanding messy, out-of-order, multi-part answers
Rules: when graduation is allowed, stall detection, when to offer text or a callback	Filling slots by calling the save tool
Events: hangups, silence timers, Google sign-in results, channel switches	Reading the user's mood from their words
Checking every value the model saves (email format, name sanity)	Warm, varied wording that never repeats
Writing the director's note each turn	Following the note naturally, without sounding scripted

3.4 The loop, every turn

1. User speaks or types.
(Voice: transcript + tags like [interrupted agent] or [3.1s pause])
2. The code builds a director's note from the slot table and signals.
3. Sonnet 5 receives: system prompt + history + the new message + the note.
4. Sonnet 5 streams a reply AND calls `save_onboarding_info(...)` when it learns something.
5. The code validates each saved value, updates the slot table, and re-checks the rules.
6. The reply is streamed to the screen (text) or to text-to-speech (voice), sentence by sentence.

An example director's note. It's added to the end of the newest user message in a tagged block, and the system prompt tells the model never to read it aloud or mention it. Sonnet 5 doesn't support mid-conversation system messages, and putting the note at the end keeps the unchanging part of the prompt cacheable.

```
<director_note>
Filled: user_name=Priya, help_topic="keeping up with school emails"
Missing: gmail
Signals: user is rushed (short answers, said "hurry")
Graduation: allowed, not yet offered
Priority: offer graduation now; mention Gmail can be connected later by text.
Keep your reply to one sentence.
</director_note>
```

3.5 A worked example

Turn	What happens
Agent	"Hey, it's Nova! Who do I have the pleasure of talking to?"
User	"Hi, I'm Maya, and honestly I just need help keeping up with my kid's school emails."
LLM	Calls <code>save_onboarding_info(user_name="Maya", help_topic="keeping up with kid's school emails")</code> . Two slots filled from one sentence.
Code	Both values pass validation. Graduation is now allowed (the help topic is known, and so is the name). The next director's note says: Gmail is missing, and connecting it is the natural way to help with school emails.
Agent	"Nice to meet you, Maya! School emails are a lot. If you connect Gmail, I can start sorting those right now. I just put a button on your screen."
User	"It's Mia, not Maya. And can we hurry up, I have a meeting in five."
LLM	Calls <code>save_onboarding_info(user_name="Mia", correction_of="user_name", sentiment="rushed")</code> .
Code	Overwrites the name. Marks the user as rushed. New note: one-sentence replies, and offer to graduate now with Gmail left for later.
Agent	"Sorry, Mia! Want to jump in now? I'll text you the Gmail link for whenever you're free."
User	"Yeah, that works."
Code	Graduated. Gmail is still missing, so the code schedules a text follow-up. The recap text is sent.

The agent never asked a question the user had already answered, handled the correction without fuss, noticed the rush, and let the user leave early while still getting what it needed. A form can't do that, and a pure prompt can't guarantee it.

4. Turn-taking: semantic turn detection and backchannel filtering

4.1 Semantic turn detection

The problem: knowing when the user is done talking. A simple silence timer either interrupts people mid-thought ("My name is... uh...") or waits too long after they finish, which makes the agent feel slow.

Method	How it decides	Result
Silence timer only (basic voice activity detection)	Waits for about 500-800ms of silence	Cuts people off, or feels laggy
Semantic turn detection	A small model judges whether what was said sounds finished	Responds fast after "I'm Sam." and waits patiently after "I'm, uh..."

Both frameworks include one: LiveKit has a turn-detector model that reads the transcript, and Pipecat has "Smart Turn," which listens to the audio. The rule to implement:

```
if turn_model says "finished"      -> respond after a short pause (about 200-300ms)
if turn_model says "not finished"  -> keep waiting, up to about 2-3 seconds of silence
if silence passes about 5 seconds -> hand off to the silence check-in logic
```

4.2 Backchannel filtering

People say "mhm," "yeah," and "right" while someone else talks, just to show they're listening. A naive agent treats these as interruptions and stops mid-sentence, which feels robotic. The rule:

```
while the agent is speaking and the user makes a sound:
  if it's a short backchannel ("mhm", "uh-huh", "yeah", "right", "okay", "got it")
    and it lasts less than about 400ms:
      keep talking                                # they're just listening
  elif it contains a stop word ("wait", "no", "stop", "actually", "hold on"):
    stop immediately and listen                    # always a real interruption
  else:
    stop, and listen                               # real speech = real interruption
```

- **Only filter while the agent is mid-sentence.** After the agent asks a question, "yeah" is an answer, not a backchannel.
- **When interrupted, save only what was actually spoken.** Cut the agent's message in the history to the words the user heard, so the model doesn't assume the user heard the rest. Check whether your framework already does this.
- Tune the 400ms threshold during testing, and record the final value and why in the README.

Decision: Both, as core features. They're the most noticeable things on the call.

5. The brain: Claude Sonnet 5

Option	Latency	Conversation quality	Tool reliability	Price per million tokens (in / out)
Claude Sonnet 5 (chosen)	Good at low effort	Excellent	Excellent	\$2 / \$10
Claude Haiku 4.5 (fallback)	Fastest Claude	Good, less nuanced	Good	\$1 / \$5
Claude Opus 5 / 5.5	Slower	Best	Best	\$5 / \$25 and \$4 / \$20
Open-weight models on fast hardware	Extremely fast	Weaker at nuance	Riskier	Cheap

How to configure it

- **Low effort** (minimal thinking), because each turn is short and speed matters more than deep reasoning.
- **Prompt caching:** put the unchanging content first (system prompt, tool definitions) and the changing content (the director's note) last. This cuts both cost and time to first token.
- **Stream the reply** and send it to text-to-speech one sentence at a time, so speech starts before the full reply is written.
- **Same model for voice and text.** Mixing models would make the channels behave differently.
- **Switch to Haiku 4.5 only if measurements show** the time to first token is too slow for the latency budget.

6. Extraction and frustration detection

6.1 Extracting information

Option	Added latency	Verdict
Tool call inside the main response (chosen)	None	Simple and accurate. The model talks and saves in one step.
Separate extraction step first	300ms or more	Slightly more accurate, but too slow for voice
Model returns JSON plus the reply	Speech waits for the JSON	Hurts latency
A small model extracts in parallel	None	Race conditions: the reply may not reflect what was just extracted

Code checks every saved value: the email must be valid, a name can't be a whole sentence, and anything saved from a low-confidence transcript is confirmed lightly with the user.

6.2 Detecting frustration

Signal	Source	Examples
What they say	The LLM, in the same call	"Just get on with it," sarcasm, sighing words, complaints
How they behave	Your code (deterministic)	Several short answers in a row, repeated interruptions, asking the same thing twice, long silences
How they sound (stretch goal)	An audio emotion model, such as Hume AI	Irritated tone, sighs. Adds a vendor and some latency.

The first two are combined into the director's note. When frustration is detected, the note tells the model to drop small talk, ask only for the single most important missing piece, and offer text or skipping.

7. Speech-to-text

Option	Notes
Deepgram streaming (chosen)	Very low latency, word timestamps, confidence scores, and keyword boosting
AssemblyAI streaming	Strong accuracy and low latency. A solid alternative.
OpenAI transcription	Accurate, but less optimized for real-time streaming
Self-hosted Whisper	Free, but real-time streaming takes real engineering. Skip it.

- **Keyword boosting:** once the user names their agent, boost that name so it's transcribed correctly every time. Do the same for the user's name once it's confirmed.
- **Confidence scores** produce the [low confidence: name] tags that trigger a light confirmation.

8. Text-to-speech and the voice bake-off

Voice is the first thing a reviewer hears and the most subjective choice in the project, so pick it with a structured test, not a guess.

Provider	Strength	Weakness
ElevenLabs	The most expressive, human-sounding voices	Pricier
Cartesia	The lowest latency, cheap, very good quality	Slightly less expressive
OpenAI TTS (baseline)	Can be steered by instructions ("sound warm and upbeat")	Fewer voices, less natural

8.1 Step 1: make voices swappable (about 45 minutes)

Build a small voice layer so that switching provider or voice is one line of config. This makes the bake-off fast and keeps the rest of the code independent of the provider.

```
VOICE = { provider: "cartesia", voice_id: "...", speed: 1.0 }  
speak(text) -> routes to the configured provider, streams audio back
```

8.2 Step 2: the shortlist

Pick 3-4 voices from ElevenLabs, 3-4 from Cartesia, and 1-2 from OpenAI as a baseline, for about 8-10 in total. Filter for voices that sound friendly and conversational, not like a narrator or announcer.

8.3 Step 3: the test script

Every voice reads the same lines, chosen to cover what the agent actually says:

Line type	Example	What it tests
Greeting	"Hey, it's Nova! Who do I have the pleasure of talking to?"	Warmth, first impression
Question	"So what's been taking up most of your time lately?"	Natural rising intonation
Empathy	"Ugh, that sounds exhausting. Let's make this quick."	Can it sound sincere, not chipper?
Excitement	"Oh nice, I can already see a few of those. Want me to sort them?"	Energy without sounding salesy
Dates and times	"You've got the dentist Thursday at 9:15 a.m."	Reading numbers and times correctly
Names	"Got it, Siobhan. And you want to call me Kai?"	Pronouncing unusual names
Long sentence	A 25-word sentence with a comma pause	Rhythm and breathing
Short filler	"Got it."	Sounds natural in isolation

8.4 Step 4: two rounds of listening

- 1 **Round 1, solo (about 30 minutes):** listen to all voices on all lines and cut down to the top 4. Drop anything robotic, mispronounced, or off-brand.

- 2 **Round 2, blind (about 30 minutes, 3-5 friends):** play the top 4 in random order without labels. Each person rates naturalness, warmth, and clarity from 1 to 5, and answers: "Which one would you want to talk to every day?"
- 3 **Round 3, in context (about 20 minutes):** run full live calls with the top 2. A voice that sounds great in a clip can sound choppy when streamed sentence by sentence.

8.5 Step 5: measure, not just listen

Voice	Natural (1-5)	Warm (1-5)	Clear (1-5)	Time to first audio (median of 10)	Pronunciation errors	Cost per 1,000 characters
...						
...						

- Put the finished scorecard in the README. "I tested 9 voices with 5 listeners and picked X because..." shows rigor and product sense.
- Optional product touch:** when the user names their agent, let them choose from your top 3 voices, each with a short sample. The voice layer makes this cheap to add, and it's a natural second personalization moment.

Decision: Choose the voice from the bake-off data, spending about 2-2.5 hours in total.

9. Voice framework

Option	Language	Turn detection	Notes
Pipecat	Python	Smart Turn (listens to audio)	Open source and very flexible. Many provider integrations.
LiveKit Agents	Python or Node.js	Turn-detector model (reads the transcript)	Open source, strong audio infrastructure, free cloud tier
Vapi / Retell (hosted)	Any	Built in	Fastest demo, but it looks outsourced and you control less of the edge cases

Decision: Pipecat if you're strongest in Python. LiveKit Agents if you want TypeScript or Node.js. Both support everything in this design.

10. Latency budget

Target: the agent starts speaking about **1 second** after the user stops.

Stage	Budget	How to hit it
Knowing the user finished	200-400ms	Semantic turn detection
Final transcript	100-300ms	Streaming speech-to-text

Stage	Budget	How to hit it
LLM first token	300-600ms	Low effort, prompt caching, a short director's note
First audio	100-200ms	Streaming text-to-speech, starting at the first sentence
Total	about 0.7-1.5 seconds	

Measure it. Log a timestamp at each stage for every turn, and report the typical (median) and slow (90th percentile) totals in the README. Very few candidates will show real latency numbers.

11. Session storage and resuming

- Store the slot table, the conversation history, and the call status in SQLite or Postgres, keyed by session ID. Keep Google sign-in tokens in a separate server-side table.
- If the server restarts during the team's review, or the user refreshes the page, the conversation resumes where it left off.
- The same history is used for both voice and text, which is what makes a hangup continue smoothly over text.

12. Stretch goals, in priority order

Stretch goal	Why it's worth it	Rough effort
Let the user choose the agent's voice	A second personalization moment. Cheap once the voice layer exists.	1-2 hours
Speech-to-speech mode behind a toggle	Shows you evaluated the frontier option hands-on, with a comparison in the README	4-6 hours
Audio emotion detection (e.g. Hume AI)	The agent reacts to tone of voice, not just words	3-4 hours
Expressive delivery	Match tone to mood (softer when the user is frustrated), if the chosen voice provider supports style controls	1-2 hours

13. Where this fits in the build plan

Project plan phase	Technical work from this document
Phase 0: setup	Choose the framework by language. Get a hello-world voice call working. Take a first latency measurement.
Phase 1: the brain (text)	Slot table, rules, director's note, save tool plus validation, Sonnet 5 configuration and caching
Phase 2: voice	Semantic turn detection, backchannel filtering, cutting interrupted replies in the history, the voice layer, and the voice bake-off. Add about 2 hours to this phase for the bake-off.
Phase 3: Gmail	Unchanged from the project plan
Phase 5: stress testing	Tune turn detection and the backchannel threshold. Collect latency numbers.
Phase 6: README	Cascaded vs. hybrid reasoning, the dialogue-management approach, the voice scorecard, and latency results

The project plan covers the schedule, scope, edge cases, and costs. This document covers how it's built and why.